Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 5

Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The aim of this experiment was to see how much the training choices around a CNN can change its behaviour. Instead of looking only at the final accuracy, the notebook compares weight initialization, regularization, batch normalization, optimizers, a few important hyperparameters, transfer learning and fine-tuning. A final 5-fold cross-validation stage was then used to choose a configuration before evaluating it on the untouched test set.

MobileNetV2 was used as the common backbone and the Oxford-IIIT Pet dataset was used for the classification task. This made it possible to keep the model architecture mostly fixed while changing one training decision at a time.

2. Background Theory

2.1 Convolutional Neural Networks

A convolutional neural network learns visual features directly from images. Early convolution layers tend to capture simple patterns such as edges and textures, while deeper layers combine these patterns into more useful shapes. In this experiment the convolutional feature extractor was supplied by MobileNetV2, and the final classifier was replaced by a layer for the 37 pet-breed classes.

For a convolution layer, the spatial output size can be written as

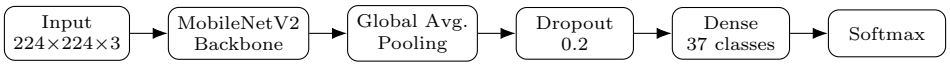
$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where N is the input size, K is the kernel size, P is the padding and S is the stride. A larger stride reduces the spatial size more quickly, while padding can be used to preserve border information.

2.2 MobileNetV2

MobileNetV2 is designed for a good balance between accuracy and computation. Its main building blocks include depthwise convolution, pointwise 1×1 convolution, inverted residual connections, linear bottlenecks, batch normalization and ReLU6. The important idea is to avoid doing a full convolution over every input and output channel when a cheaper factorized operation can do most of the work.

The model used here starts with a $224 \times 224 \times 3$ RGB image. The pretrained MobileNetV2 base produces a $7 \times 7 \times 1280$ feature map. Global average pooling reduces this to 1280 values, followed by dropout and a 37-unit softmax classifier.



2.3 Weight Initialization

Weights have to start somewhere before training begins. Four choices were tested: zeros, a small random normal distribution, Xavier/Glorot initialization and He initialization. In a fully trainable neural network, zero initialization can make neurons learn the same thing. Here, however, the MobileNetV2 feature extractor was frozen and only the final classifier was newly initialized. That detail matters when interpreting the surprisingly strong zero-initialization result.

2.4 Regularization

Regularization is intended to reduce the tendency of a model to memorize the training set. The experiment compared a plain classifier with L2 regularization, dropout and batch normalization. One useful indicator was the gap between training accuracy and validation accuracy. A smaller gap is not automatically better, but it gives a quick view of how far the model has separated its training performance from its validation performance.

2.5 Batch Normalization

For a mini-batch $x_1, \ldots, x_m$, batch normalization first calculates

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} x_i, \qquad \sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2.$$

The normalized value is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}},$$

and the learnable scale and shift give

$$y_i = \gamma \hat{x}_i + \beta.$$

For the simple example $x = [2, 4, 6, 8]$, the mean is 5 and the variance is 5. Therefore the normalized values are approximately

$$[-1.342, -0.447, 0.447, 1.342].$$

This example shows the basic effect clearly: the values are brought to a comparable scale before the learned scale and shift are applied.

2.6 Optimizers

The optimizer decides how the model parameters are changed from the calculated gradients. SGD uses the current gradient directly, while Momentum keeps part of the previous update. RMSProp adapts the update using a moving average of squared gradients, and Adam combines adaptive scaling with momentum-like estimates. Their practical behaviour can be quite different even when the model and dataset are unchanged.

2.7 Transfer Learning and Fine-Tuning

With transfer learning, a network trained on a large dataset can provide useful visual features for another task. In the feature-extraction stage, the MobileNetV2 base was frozen and only the new classifier was trained. In fine-tuning, the upper part of the pretrained network was unfrozen and updated using a much smaller learning rate of 10^{-5}. The smaller rate helps avoid changing useful pretrained features too aggressively.

2.8 K-Fold Cross-Validation

For five folds, the mean validation accuracy is

$$\bar{A} = \frac{1}{5}\sum_{i=1}^{5} A_i,$$

and the standard deviation used in the notebook is

$$SD = \sqrt{\frac{1}{5}\sum_{i=1}^{5}(A_i - \bar{A})^2}.$$

The result is reported as $\bar{A} \pm SD$. The independent test set is kept outside this selection process so that it remains a genuine final check.

3. Dataset Description

The experiment uses the Oxford-IIIT Pet Dataset. It contains images from 37 cat and dog breeds. The notebook uses the official train/validation split and the official test split rather than mixing the two.

Property	Value
Dataset	Oxford-IIIT Pet Dataset
Image type	RGB
Input size	$224 \times 224 \times 3$
Number of classes	37
Official train/validation images	3680
Training samples used	2944
Validation samples used	736
Independent test samples	3669
Batch size	32
Normalization	MobileNetV2 preprocessing
Random seed	42

The input images are resized to 224×224 and passed through the MobileNetV2 preprocessing function. The resulting batch shown by the notebook has shape $(32, 224, 224, 3)$ with values in the range $[-1, 1]$.

4. Experimental Procedure

The work was carried out in stages. The same general data pipeline was reused so that the comparisons were not affected by changes in image size or preprocessing.

1. Download and extract the official Oxford-IIIT Pet images and annotations.
2. Split the official train/validation portion into 2944 training and 736 validation images.
3. Resize images to 224×224 and apply MobileNetV2 preprocessing.
4. Build a MobileNetV2 classifier with the pretrained ImageNet backbone frozen.
5. Compare four weight initialization methods for the new classifier.
6. Compare no regularization, L2, dropout and batch normalization.
7. Run a separate batch-normalization comparison.
8. Compare SGD, Momentum, RMSProp and Adam.
9. Tune learning rate, batch size and dropout rate using controlled trials.
10. Compare feature extraction with partial fine-tuning.
11. Test four promising configurations with 5-fold cross-validation.
12. Retrain the selected configuration on all 3680 training/validation images and evaluate it once on the 3669-image independent test set.

5. MobileNetV2 Model Setup

The base network was loaded with ImageNet weights and frozen. Global average pooling was added instead of flattening the full feature map. A dropout layer with rate 0.2 and a dense softmax layer with 37 outputs completed the classifier.

The model summary reported 2,305,381 parameters in total. Of these, 47,397 were trainable and 2,257,984 were frozen. The output of the backbone was $7 \times 7 \times 1280$, which becomes a 1280-element vector after global average pooling.

Listing 1: Core MobileNetV2 model construction

```
base_model = keras.applications.MobileNetV2(
    input_shape=(224, 224, 3),
    include_top=False,
    weights="imagenet"
)
base_model.trainable = False

inputs = keras.Input(shape=(224, 224, 3))
x = base_model(inputs, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.2)(x)
outputs = layers.Dense(37, activation="softmax")(x)

```

```
model = keras.Model(inputs, outputs)
```

6. Weight Initialization Study

Four initializers were applied only to the new classification layer. All four models used Adam with a learning rate of 0.001 and were trained for 10 epochs.

Initialization	Final Train Loss	Final Val. Accuracy	Best Val. Accuracy
Zero	0.0334	92.39%	92.80%
Random Normal	0.0352	92.80%	**93.07%**
Xavier/Glorot	0.0339	91.98%	92.26%
He	0.0351	92.39%	92.39%

The random initialization gave the highest validation accuracy, 93.07%. The difference is not huge, but it was the best of the four runs. The zero-initialized classifier also performed well because the large pretrained feature extractor was not being re-initialized or trained from scratch.

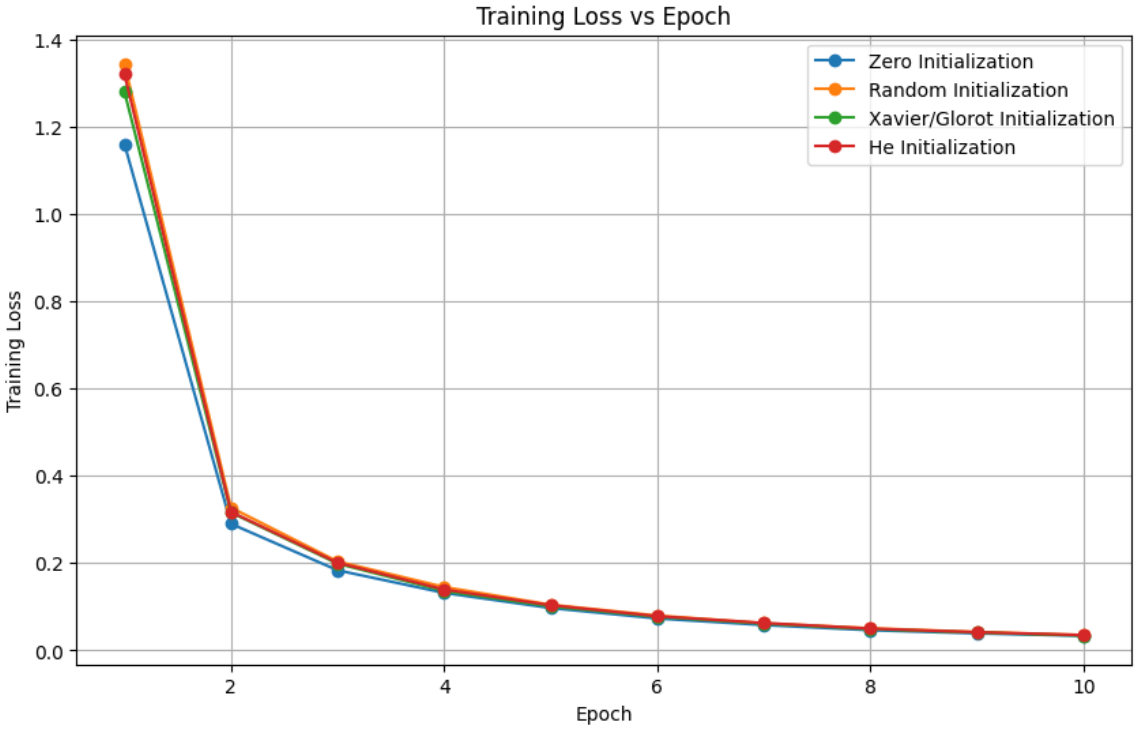


Figure 1: Training loss for the four initialization strategies.

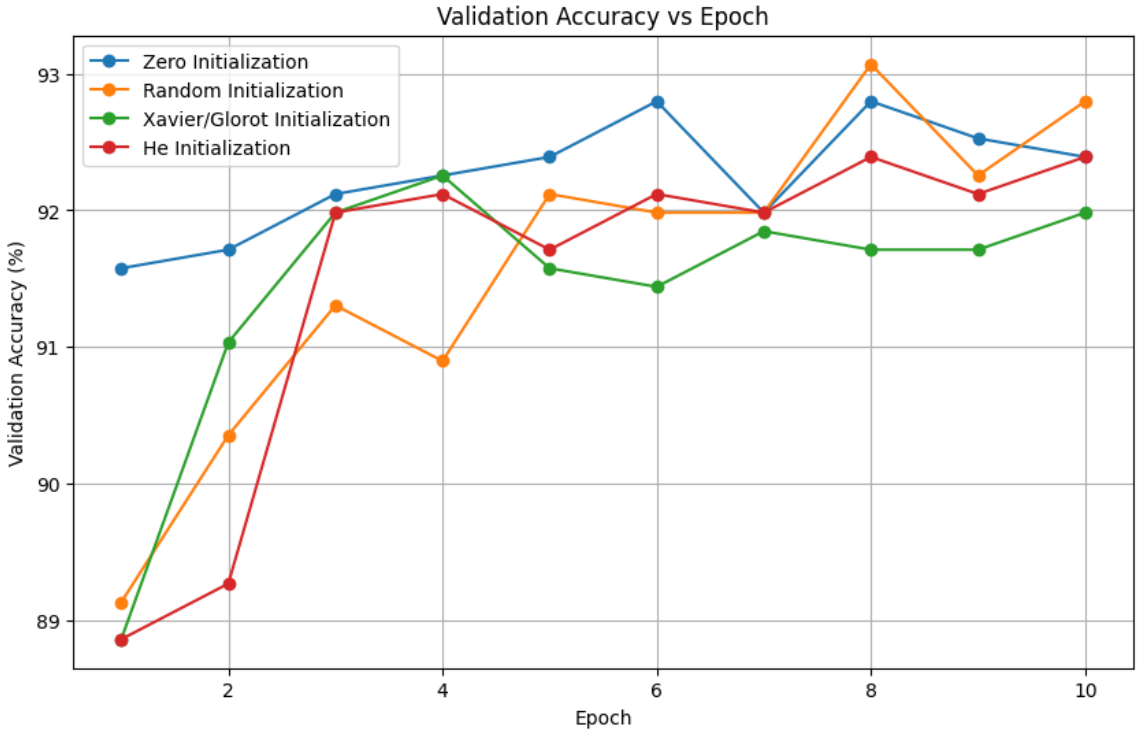


Figure 2: Validation accuracy for the four initialization strategies.

7. Regularization and Overfitting Study

The next comparison looked at how different forms of regularization changed the training and validation behaviour. The final results from the notebook are summarized below.

Method	Final Train Acc.	Final Val. Acc.	Best Val. Acc.	Gap
No Regularization	99.93%	91.71%	92.12%	8.22%
L2 Regularization	99.93%	91.85%	92.12%	8.08%
Dropout	97.62%	91.85%	91.85%	**5.77%**
Batch Normalization	99.93%	91.98%	91.98%	7.95%

The most obvious change came from dropout. Its training accuracy was lower, but the training–validation gap was also smaller. That is the expected direction for a regularizer: the model gives up some training fit in exchange for less separation between training and validation behaviour. It did not, however, give the highest validation accuracy in this run.

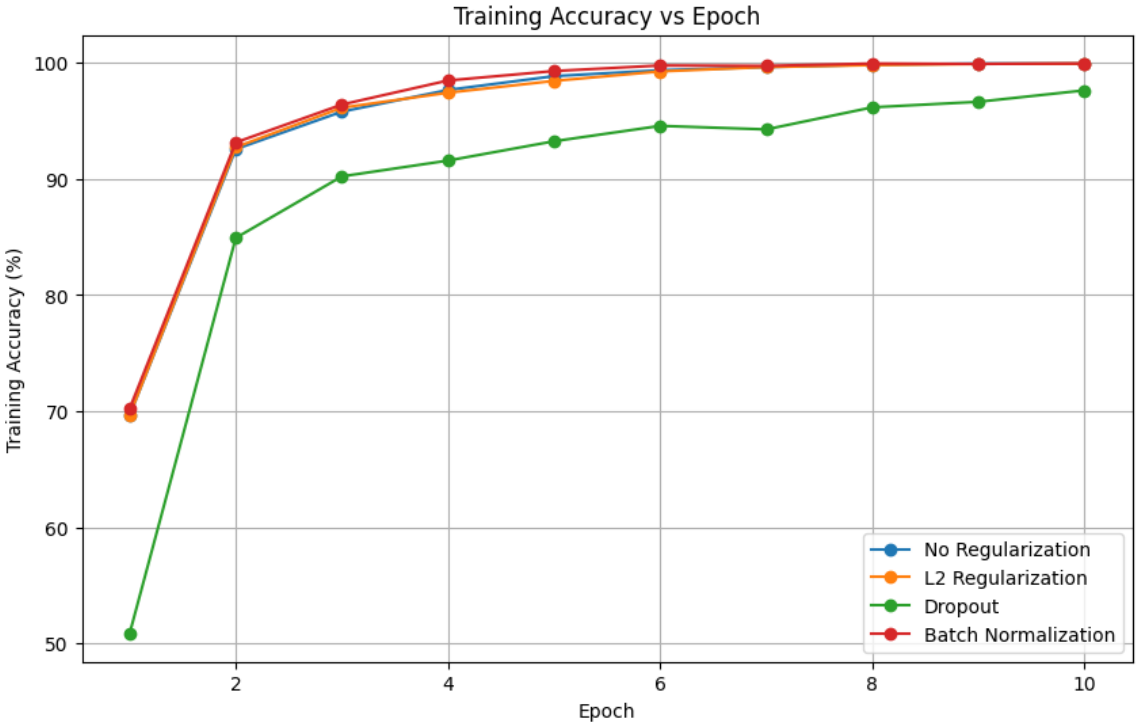


Figure 3: Training accuracy during the regularization comparison.

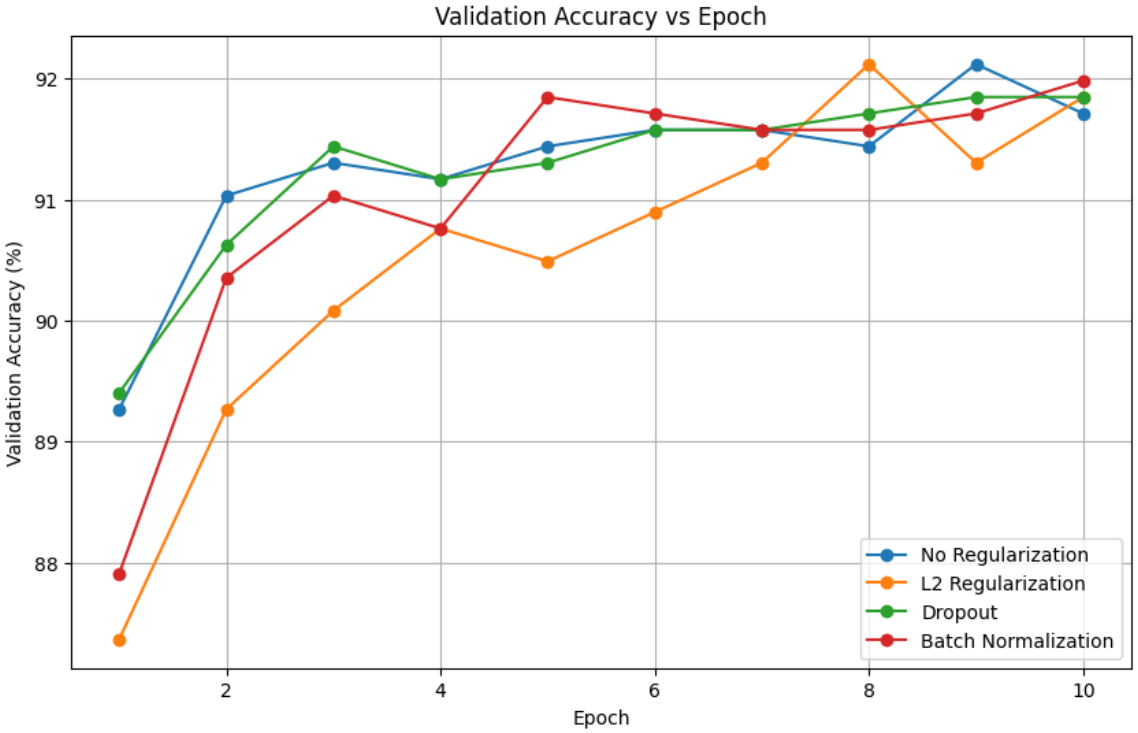


Figure 4: Validation accuracy during the regularization comparison.

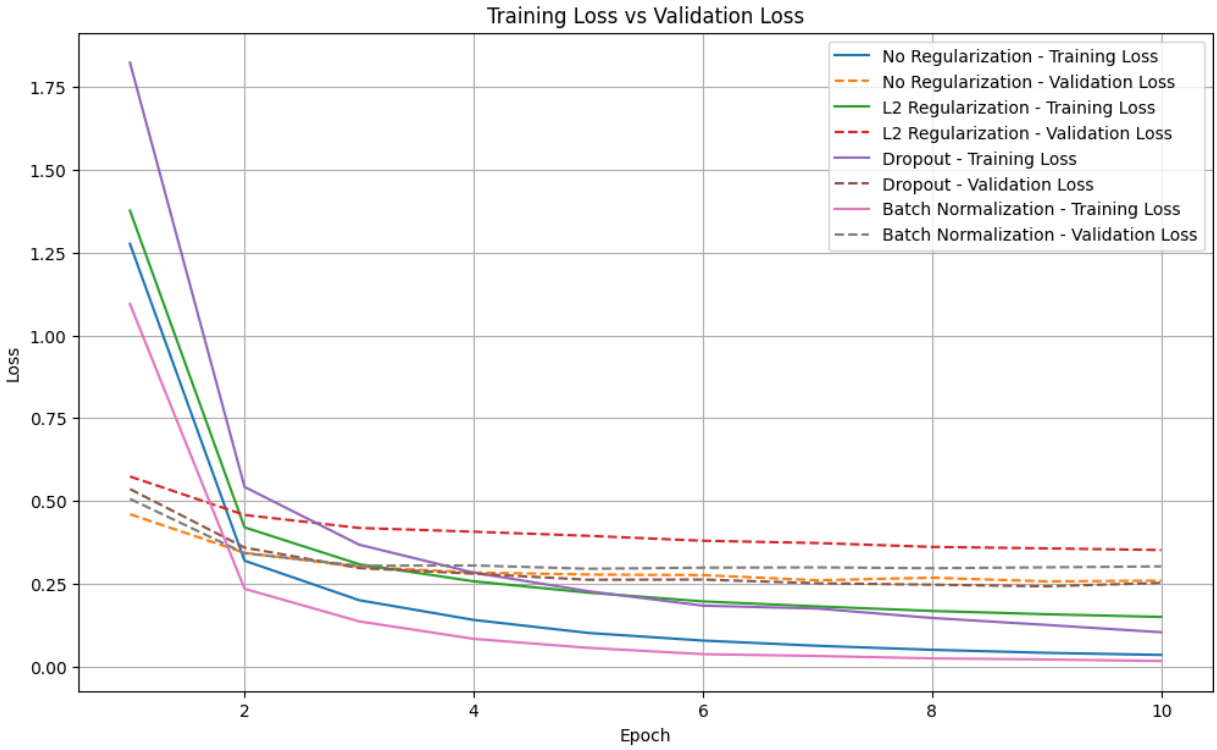


Figure 5: Training and validation loss for the regularization methods.

8. Batch Normalization Study

A separate experiment compared the same classifier with and without batch normalization. The best validation accuracy without BN was 92.12%, while the BN model reached 92.26%. The improvement is only 0.14 percentage points, so it is better described as a small improvement rather than a dramatic change.

Model	Final Validation Accuracy	Best Validation Accuracy
Without BN	92.12%	92.12%
With BN	91.44%	**92.26%**

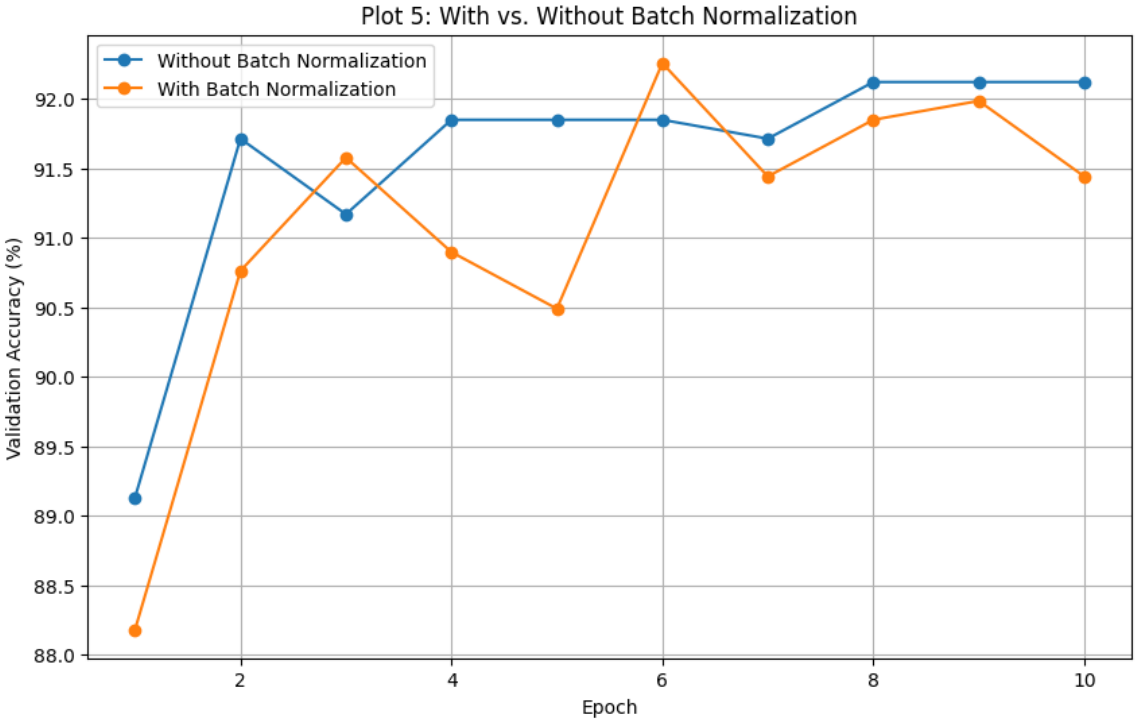


Figure 6: Validation accuracy with and without batch normalization.

One detail is worth pointing out. The final accuracy of the BN run was lower than its best validation accuracy, which means the best checkpoint in terms of validation accuracy occurred before the last epoch. This is another reason not to judge a training run from its final epoch alone.

9. Optimization Algorithm Comparison

SGD, Momentum, RMSProp and Adam were compared using the same basic classifier and 10 training epochs.

Optimizer	Final Loss	Best Val. Accuracy	Reported Epoch	Time (s)
SGD	1.8872	66.30%	10	105.35
Momentum	0.3070	91.44%	10	113.07
RMSProp	**0.0194**	**93.21%**	10	112.00
Adam	0.0348	92.80%	10	107.09

RMSProp was clearly the strongest of these four runs. It reached 93.21% validation accuracy and also had the lowest final training loss. SGD was much weaker in this setup, ending at 66.30% validation accuracy.

The notebook labels epoch 10 as the convergence point for all four optimizers. Since every run was deliberately limited to 10 epochs, this should not be interpreted as proof that SGD converged faster; it is more accurate to say that the runs all reached the end of the scheduled training period.

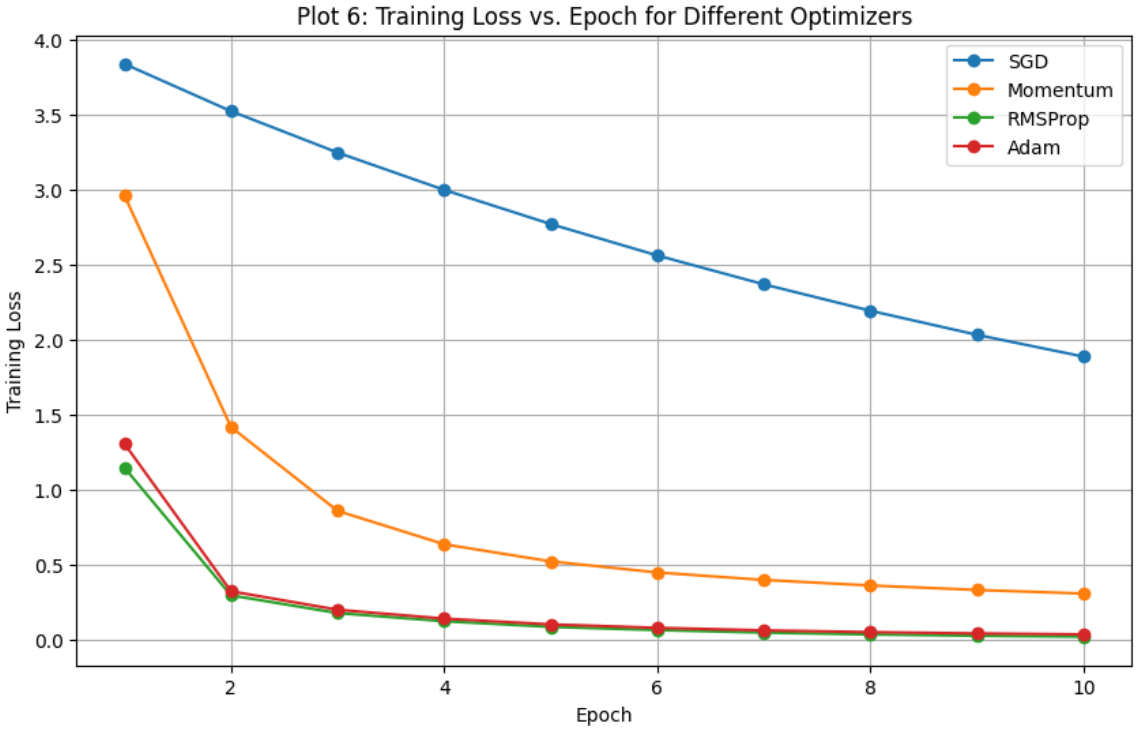


Figure 7: Training loss for the four optimizers.

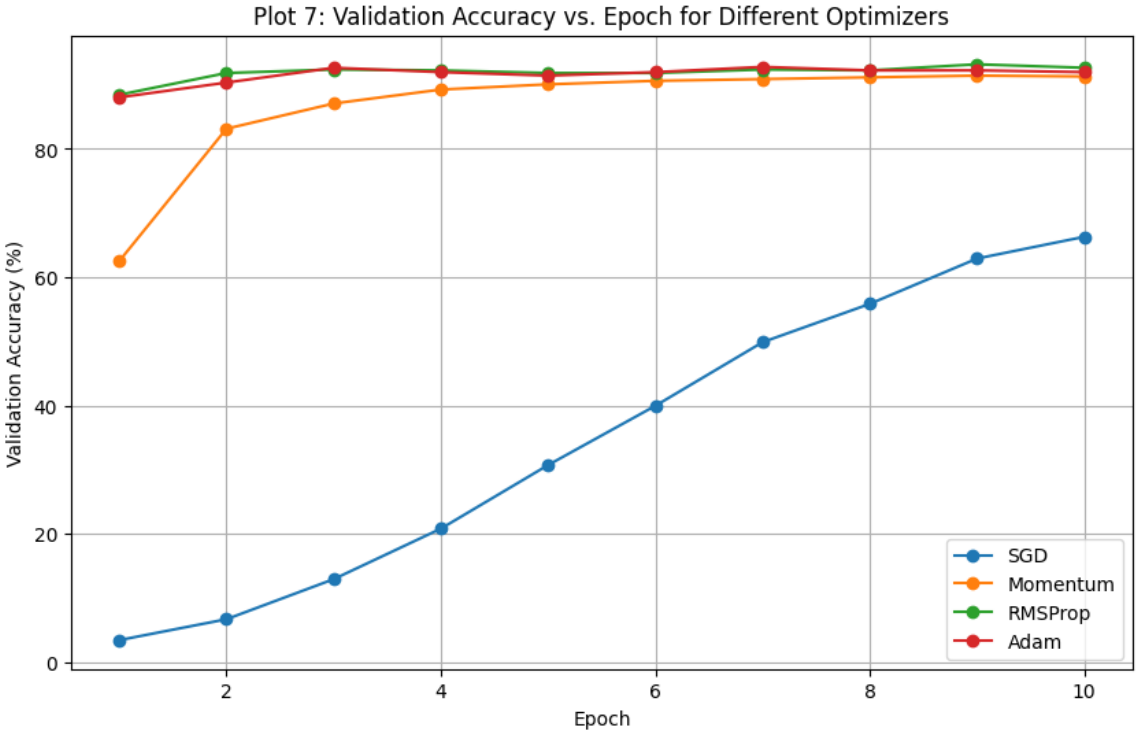


Figure 8: Validation accuracy for the four optimizers.

10. CNN Hyperparameter Tuning

Three hyperparameters were varied separately while the other settings were kept fixed for each small study.

Hyperparameter	Values tested	Best result
Learning rate	0.001, 0.0001	0.001 → 91.17%
Batch size	16, 32, 64	16 → 87.23%
Dropout rate	0, 0.25, 0.5	0 → 85.19%

The learning-rate comparison showed a clear difference: 0.001 gave 91.17% while 0.0001 gave 84.38%. For batch size, 16 was the best of the tested values, while the result dropped as the batch size increased to 64. In the dropout study, the run with no dropout was the strongest of the three tested settings. These results should be read as outcomes of the particular controlled trials rather than universal rules for every CNN.

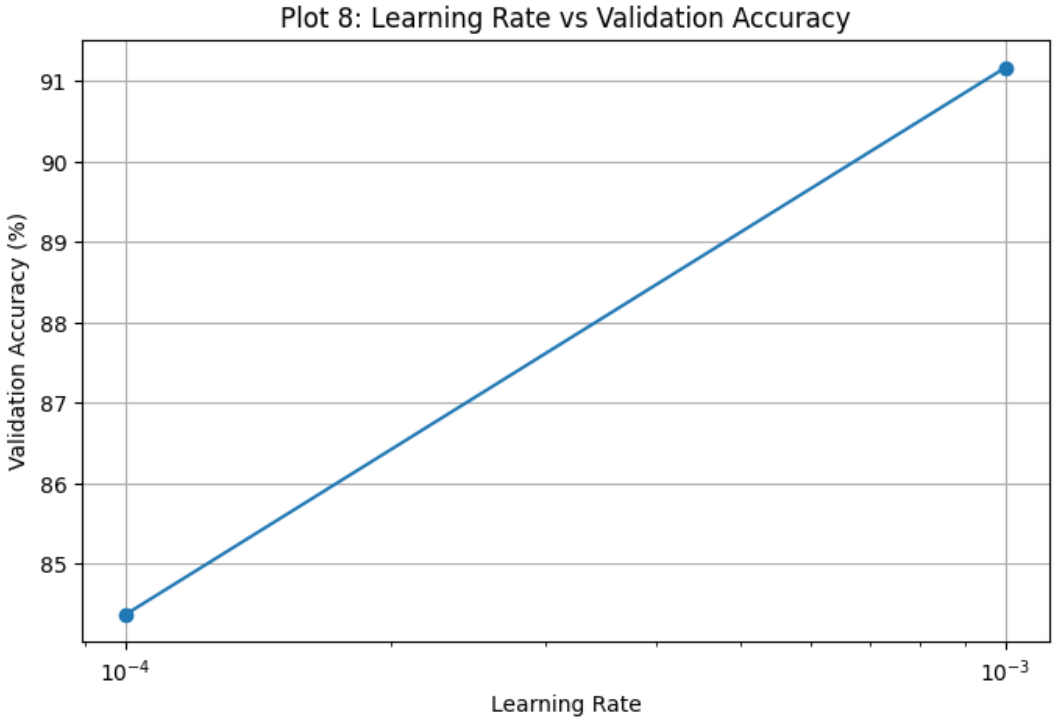


Figure 9: Learning-rate tuning results.

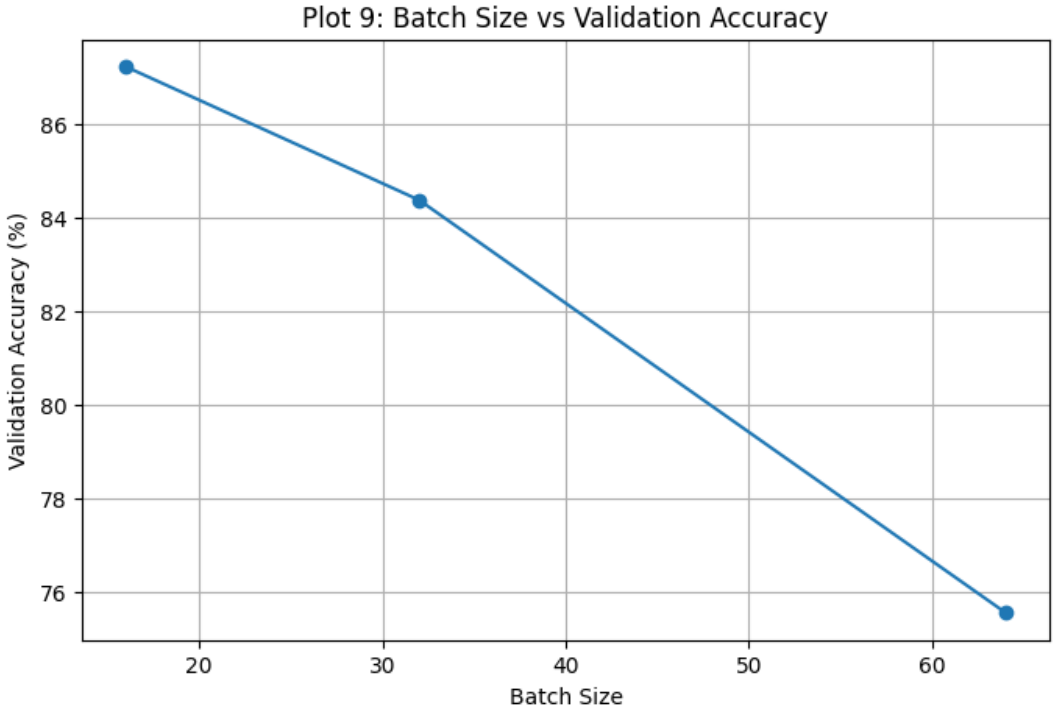


Figure 10: Batch-size tuning results.

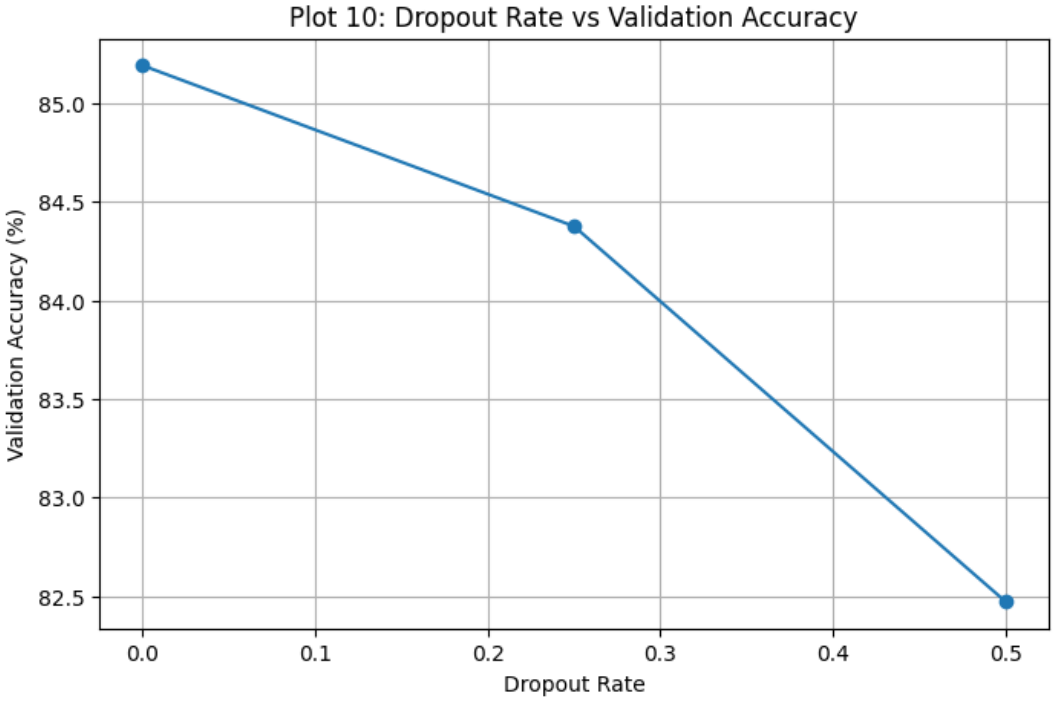


Figure 11: Dropout-rate tuning results.

11. Transfer Learning and Fine-Tuning

Two training modes were compared. In feature extraction, the MobileNetV2 backbone stayed frozen and only the new classifier was trained. In fine-tuning, the upper 36 MobileNetV2 layers were made trainable while 118 remained frozen. The fine-tuning learning rate was reduced to 10^{-5}.

Training mode	Best Validation Accuracy	Final Validation Loss
Feature extraction	92.39%	0.2431
Fine-tuning	**92.80%**	0.3173

Fine-tuning improved the best validation accuracy by 0.41 percentage points. Interestingly, its final validation loss was higher than the feature-extraction run even though its best accuracy was higher. Accuracy and cross-entropy loss measure different things, so this is not necessarily a contradiction.

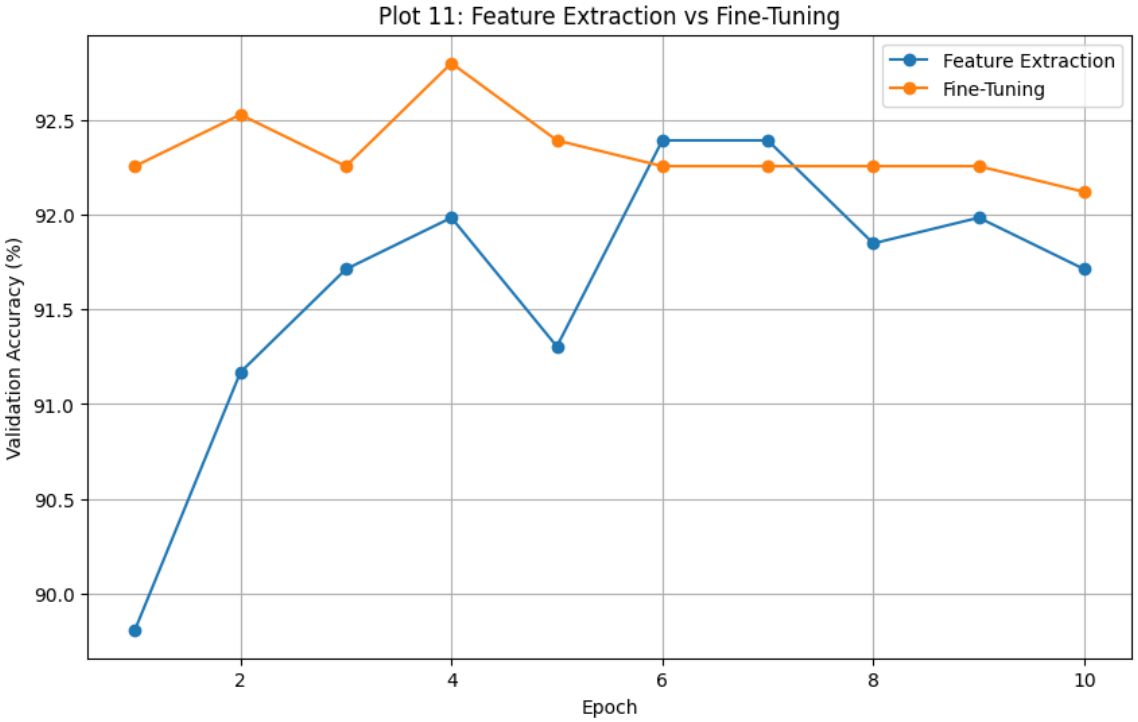


Figure 12: Validation accuracy for feature extraction and fine-tuning.

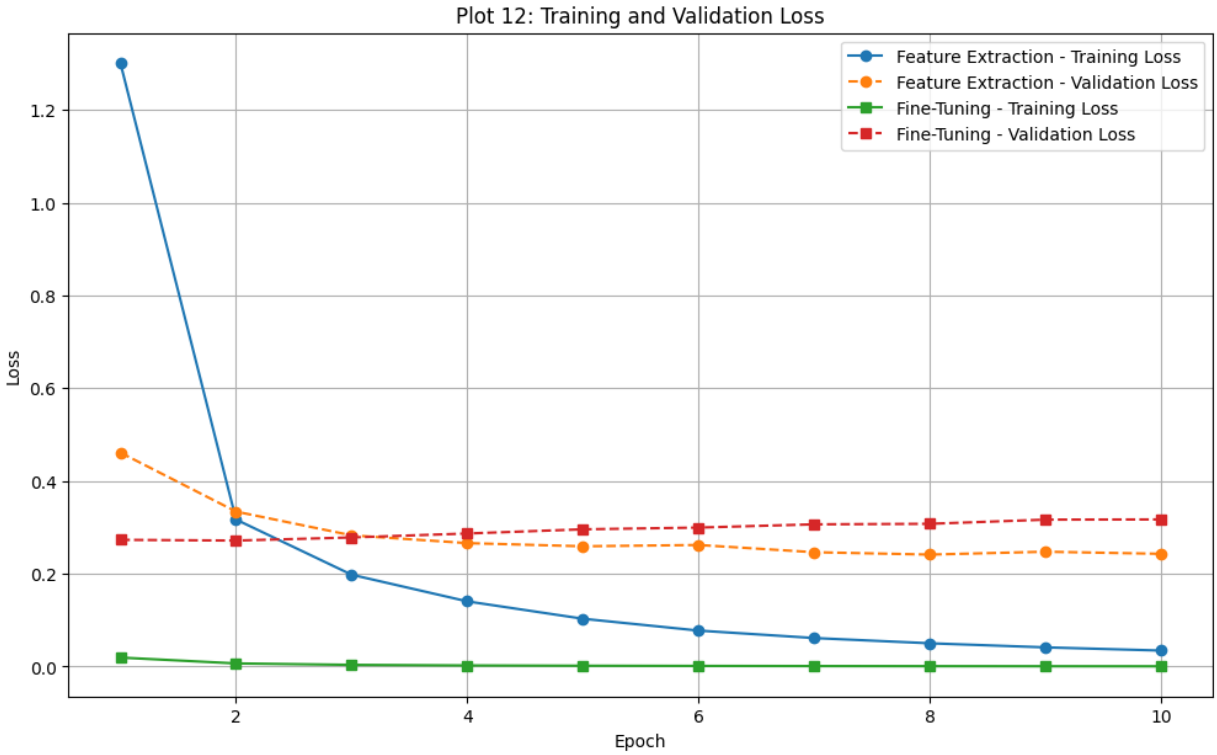


Figure 13: Training and validation loss during transfer learning and fine-tuning.

12. Five-Fold Cross-Validation

After the earlier studies, four configurations were taken into the cross-validation stage. The test set was not used here. Each configuration was trained for three epochs per fold to keep the cross-validation experiment practical in Colab.

Config.	Main settings	Mean Accuracy	SD
C1	Adam, lr=0.001, batch=32, dropout=0.25	**90.30%**	**1.30%**
C2	Adam, lr=0.0001, batch=32, dropout=0.25	72.15%	1.52%
C3	Adam, lr=0.0001, batch=32, dropout=0.50	66.82%	1.53%
C4	SGD, lr=0.001, batch=32, dropout=0.25	17.66%	3.25%

The five fold accuracies for C1 were 92.12%, 88.72%, 89.13%, 91.44% and 90.08%. Their mean was 90.30% with a standard deviation of 1.30%. C1 was therefore selected for final retraining.

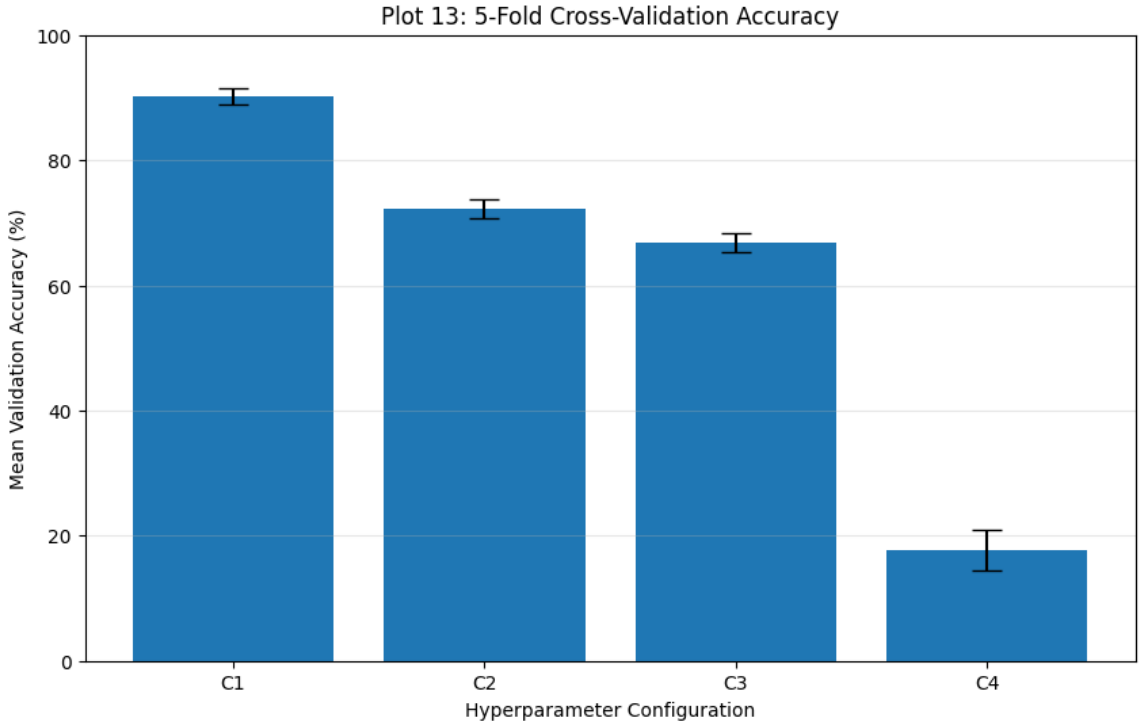


Figure 14: Mean five-fold cross-validation accuracy with standard-deviation error bars.

13. Final Model Training and Test Evaluation

The selected C1 configuration was retrained using all 3680 official train/validation images. The 3669-image official test set was left untouched until this stage. The final model again had 2,305,381 parameters.

The final training run reached 99.54% training accuracy after 10 epochs. On the independent test set, the model achieved 90.57% accuracy.

Metric	Value
Mean CV Accuracy	90.30%
CV Standard Deviation	1.30%
Test Accuracy	**90.57%**
Precision	90.74%
Recall	90.57%
F1-score	90.52%
Training Time	101.29 s (1.69 min)
Number of Parameters	2,305,381

The test accuracy is very close to the cross-validation mean: the difference is about 0.27 percentage points. That is a useful sign that the selected configuration did not collapse when moved from the cross-validation setting to the independent test set.

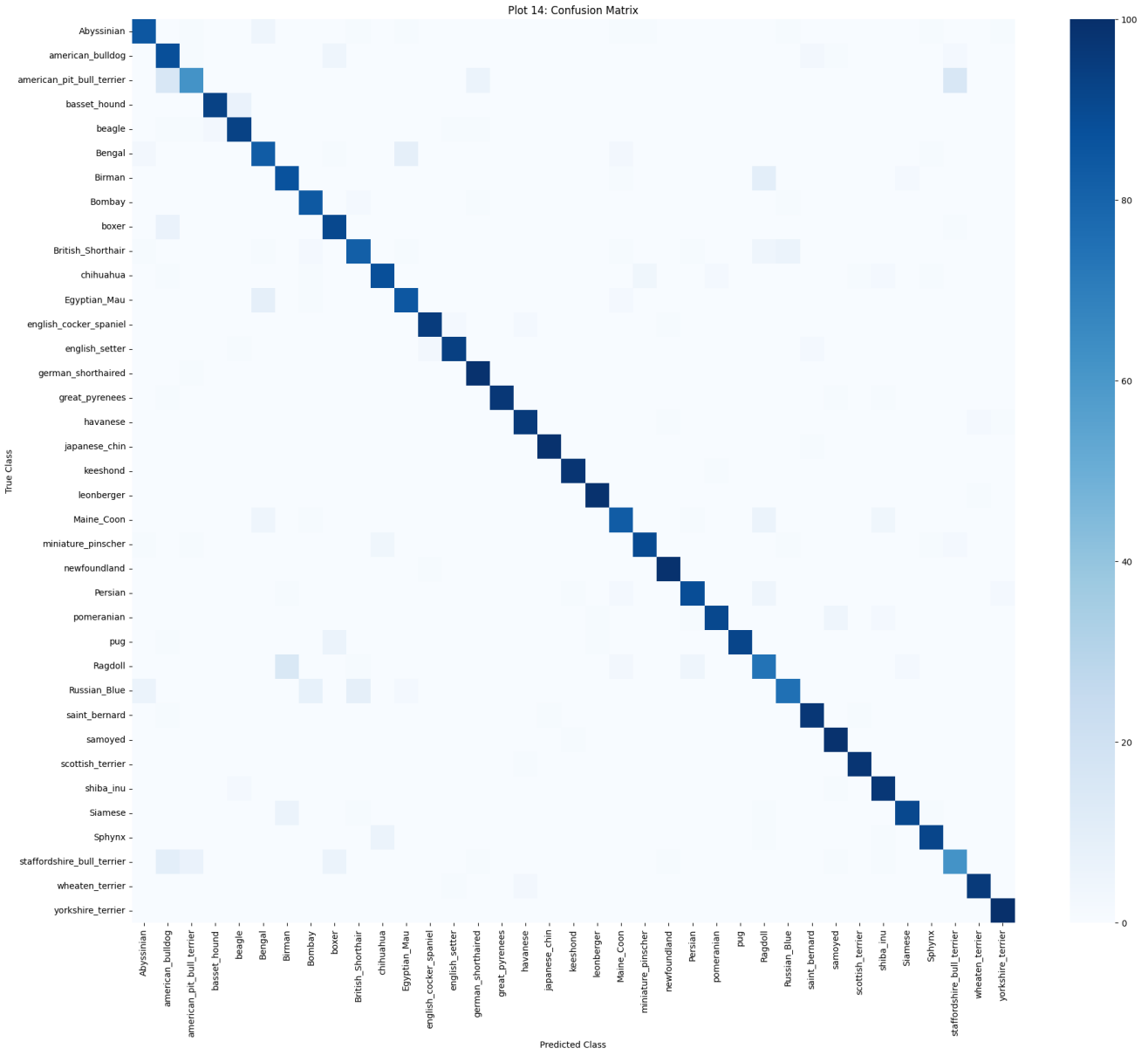


Figure 15: Confusion matrix for the final model on the independent test set.

The strongest class-wise result was for `yorkshire_terrier`, which reached 100% recall in the test set. Other classes with 99% recall included `newfoundland`, `leonberger`, `japanese_chin` and `samoyed`. The weakest recall among the listed classes was 62% for `american_pit_bull_terrier`.

The largest reported confusions were between visually similar breeds. There were 16 `american_pit_bull_terrier` images predicted as `staffordshire_bull_terrier`, another 16 predicted as `american_bulldog`, and 15 `Ragdoll` images predicted as `Birman`. These errors are consistent with the fact that some breeds share very similar body shape, coat colour and facial features.

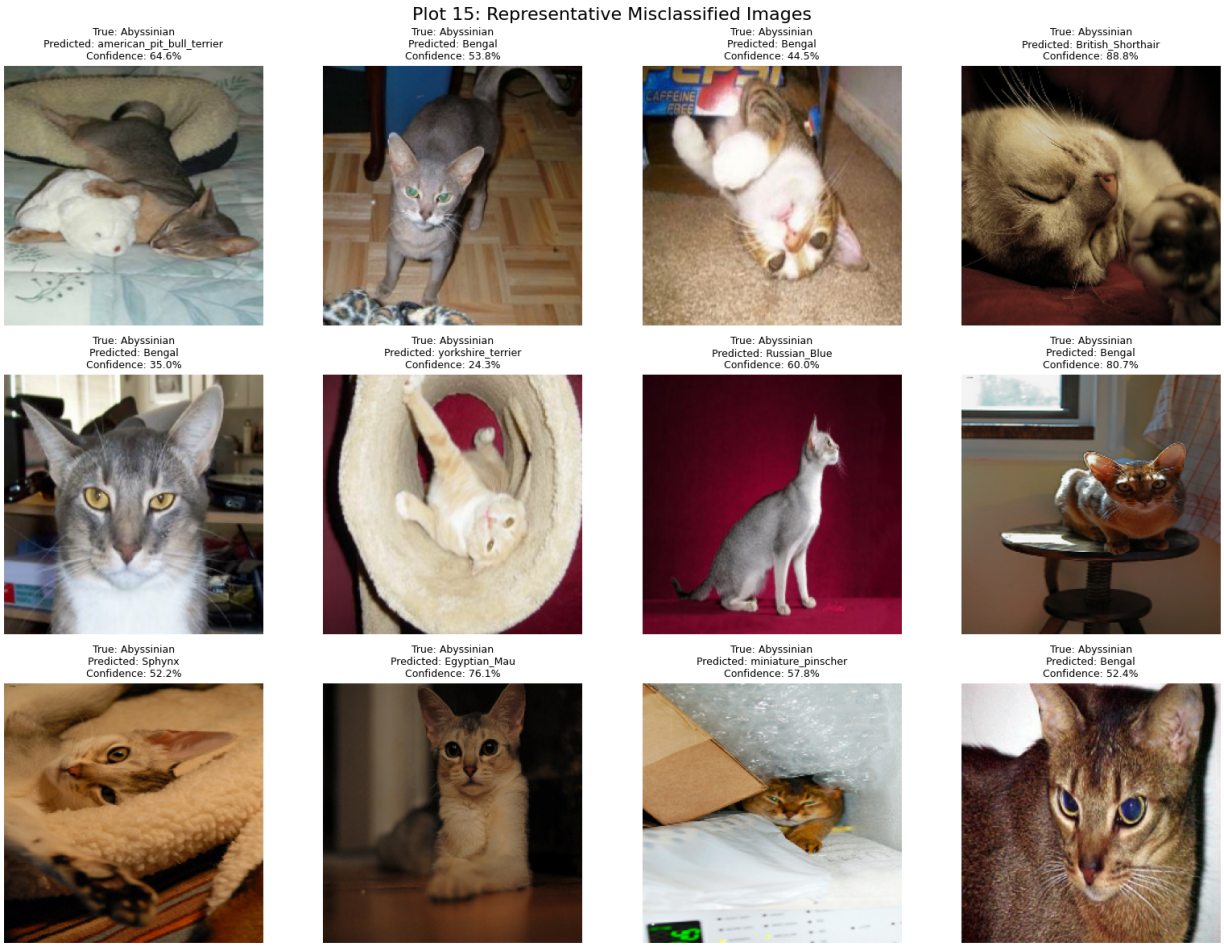


Figure 16: Examples of images misclassified by the final model.

A total of 346 test images were misclassified. The displayed examples make the limitation easier to see than a single accuracy value: several errors involve breeds whose appearance overlaps strongly with another class.

14. Performance Summary

Study	Best Validation Accuracy	Main Observation	Selected?
Initialization	93.07%	Random initialization performed best	Yes
Regularization	92.12%	Dropout reduced the train–val gap	No
Batch Normalization	92.26%	Small improvement over no BN	No
Optimization	93.21%	RMSProp performed best	Yes
Learning rate tuning	91.17%	0.001 outperformed 0.0001	Yes
Batch size tuning	87.23%	16 was best of tested values	No
Dropout tuning	85.19%	No dropout was best in this trial	No
Fine-tuning	92.80%	Small gain over feature extraction	Yes
5-fold CV	$90.30 \pm 1.30\%$	C1 was most reliable of four configs	Yes
Final test	90.57%	Good agreement with CV mean	Final

15. Source Code Used

The notebook contains the complete runnable implementation. The following excerpts show the main logic used in the experiment.

15.1 Data preprocessing

Listing 2: Image preprocessing used for MobileNetV2

```
def preprocess_image(image_path, label):
    image = tf.io.read_file(image_path)
    image = tf.image.decode_jpeg(image, channels=3)
    image = tf.cast(image, tf.float32)
    image = tf.image.resize(image, (224, 224))
    image = tf.keras.applications.mobilenet_v2.preprocess_input(image)
    return image, label
```

15.2 Per-experiment initialization

Listing 3: Changing only the classifier initializer

```
initialization_methods = {
    "Zero Initialization": initializers.Zeros(),
    "Random Initialization": initializers.RandomNormal(
        mean=0.0, stddev=0.05),
    "Xavier/Glorot Initialization": initializers.GlorotUniform(seed=42),
    "He Initialization": initializers.HeNormal(seed=42)
}
```

15.3 Optimizer setup

Listing 4: Optimizer selection used in the comparison

```
def get_optimizer(name, learning_rate):
    if name == "SGD":
        return keras.optimizers.SGD(learning_rate=learning_rate)
    if name == "Momentum":
        return keras.optimizers.SGD(
            learning_rate=learning_rate, momentum=0.9)
    if name == "RMSProp":
        return keras.optimizers.RMSprop(learning_rate=learning_rate)
    return keras.optimizers.Adam(learning_rate=learning_rate)
```

15.4 Final evaluation

Listing 5: Metrics reported for the independent test set

```
y_pred = model.predict(test_ds)
y_pred = np.argmax(y_pred, axis=1)

accuracy = accuracy_score(test_labels, y_pred)
precision = precision_score(
    test_labels, y_pred, average="weighted", zero_division=0)
recall = recall_score(
    test_labels, y_pred, average="weighted", zero_division=0)
f1 = f1_score(
    test_labels, y_pred, average="weighted", zero_division=0)
```

16. Discussion

The experiment gave a useful picture of how much the training setup matters even when the backbone is fixed. The first important point is that the pretrained feature extractor did most of the heavy lifting. Only a small classification layer was trained in the baseline model, which explains why the model could reach validation accuracies around 92–93% after only a few epochs.

The initialization experiment also needs to be interpreted in that context. Random initialization gave the best validation accuracy at 93.07%, but the zero-initialized classifier was close behind. This would be surprising if the whole CNN had been initialized to zero and trained from scratch. Since only the final layer was initialized, the usual symmetry problem is much less severe here.

Regularization showed a different kind of effect. Dropout reduced the overfitting gap from 8.22% to 5.77%, but its validation accuracy was not the best. This is a good reminder that reducing the training–validation gap and maximizing validation accuracy are related goals, not identical ones.

The optimizer comparison was the clearest result. RMSProp reached 93.21%, ahead of Adam at 92.80%, Momentum at 91.44% and SGD at 66.30%. The gap between SGD and the adaptive optimizers was large enough that it cannot be dismissed as a tiny fluctuation from the run.

Fine-tuning produced a smaller improvement, from 92.39% to 92.80% in best validation accuracy. The gain was modest, but the experiment still demonstrates why unfreezing the upper pretrained layers can help: the features learned on ImageNet can be adjusted toward the pet-breed task instead of being kept completely fixed.

Finally, the cross-validation step was important because it changed the question from "which run looked best?" to "which configuration stayed reasonably good across different folds?". C1 achieved 90.30 ± 1.30%, which is substantially better than the other three tested configurations. The final independent test accuracy of 90.57% was close to the cross-validation mean, giving some confidence that the selected configuration generalizes reasonably well.

17. Conclusion

This experiment implemented and compared several practical choices involved in training an image classifier with MobileNetV2. The Oxford-IIIT Pet dataset was processed using a fixed image pipeline, and the pretrained backbone was first used as a frozen feature extractor.

Among the initialization methods, random initialization of the new classifier gave the highest validation accuracy of 93.07%. In the regularization study, dropout produced the smallest training–validation gap, although it did not produce the highest validation accuracy. RMSProp was the strongest optimizer in the direct comparison, reaching 93.21% validation accuracy.

The hyperparameter trials showed that the tested learning rate of 0.001 was better than 0.0001, while batch size 16 and dropout rate 0 gave the best results within their respective small searches. Fine-tuning the upper MobileNetV2 layers improved the best validation accuracy slightly to 92.80%.

The final cross-validation stage selected C1, with a mean accuracy of 90.30 ± 1.30%. After retraining on the complete training/validation data, the model obtained 90.57% accuracy, 90.74% weighted precision, 90.57% weighted recall and 90.52% weighted F1-score on the independent test set.

Overall, the experiment showed that there is no single training choice that can be judged in isolation. Accuracy, stability across folds, training behaviour and computational cost all give useful information when deciding which configuration to keep.

18. References

1. M. Sandler, A. Howard, M. Zhu, A. Zhmoginov and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," *CVPR*, 2018.
2. O. M. Parkhi, A. Vedaldi, A. Zisserman and C. V. Jawahar, "Cats and Dogs," *IEEE Conference on Computer Vision and Pattern Recognition*, 2012.
3. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
4. S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," *ICML*, 2015.
5. TensorFlow and Keras documentation, used for the MobileNetV2, preprocessing, model construction and training APIs.

19. Source Notebook

The results and figures in this report were taken from the supplied Google Colab/Jupyter notebook for Experiment 5. The notebook contains the complete executable implementation and the recorded outputs used to prepare the tables and figures in this report.

20. GitHub Repository

The complete implementation and related Deep Learning laboratory work are maintained in the following GitHub repository:

`https://github.com/ReshmanthCH/deep-learning`